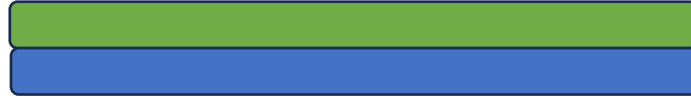


“INGENIERÍA DE SOFTWARE”



**L. SISTEMAS COMPUTACIONALES
ADMINISTRATIVOS**



Universidad Veracruzana
Facultad de Contaduría y
Administración
Xalapa

TEMA: PROYECTO INTEGRADOR

MAESTRA: MAYRA MINERVA MENDEZ ANOTA

**ALUMNOS: JUÁREZ MIJANGOS SERGIO
ROMERO RIVERA PABLO**

ÍNDICE

INTRODUCCIÓN	3
JUSTIFICACIÓN.....	4
HISTORIA DEL USUARIO	6
DIAGRAMAS UML INICIAL Y MEJORADO.....	13
Matriz de casos de prueba y evidencia de caja negra.....	15
Análisis métrico del software.....	17
Análisis crítico.....	26
Conclusión.....	27
Anexo 1	28

INTRODUCCIÓN

En la actualidad, la información financiera constituye una herramienta esencial para la toma de decisiones dentro de cualquier organización. Analizar de manera eficiente los estados financieros permite conocer la situación económica de una empresa, identificar fortalezas y detectar áreas de oportunidad que contribuyan a mejorar su desempeño.

El presente proyecto consiste en el desarrollo de un Sistema de Análisis Financiero, implementado en el lenguaje de programación Java bajo el patrón de arquitectura Modelo-Vista-Controlador (MVC). El sistema tiene como finalidad facilitar el cálculo e interpretación de diversos indicadores financieros, permitiendo al usuario ingresar información contable de manera sencilla y obtener resultados confiables en tiempo real.

Entre las principales funcionalidades del sistema se encuentran el cálculo de indicadores como ROA (Return on Assets), ROE (Return on Equity), Margen Neto, Prueba Ácida y Nivel de Endeudamiento, así como la generación de reportes en formato PDF y la visualización de gráficas que facilitan el análisis e interpretación de los resultados obtenidos.

Además, durante el desarrollo del proyecto se aplicaron buenas prácticas de ingeniería de software, utilizando una arquitectura organizada, pruebas unitarias con JUnit para validar el correcto funcionamiento del sistema y herramientas de análisis de calidad de código, con el propósito de garantizar la confiabilidad, mantenibilidad y calidad del software.

Este proyecto busca ofrecer una solución práctica que apoye el análisis financiero mediante la automatización de cálculos, la presentación clara de la información y el uso de tecnologías que fortalecen el proceso de desarrollo de software.

JUSTIFICACIÓN

El proyecto fue desarrollado con la finalidad de automatizar el registro, análisis y visualización de información financiera de empresas, permitiendo calcular indicadores financieros de manera rápida, precisa y organizada.

El sistema facilita la captura de datos financieros como:

- **activos,**
- **pasivos,**
- **capital,**
- **ventas,**
- **costos,**
- **gastos e impuestos,**

para posteriormente generar resultados y razones financieros importantes que apoyen la toma de decisiones.

Además, el sistema permite:

- Visualizar gráficas financieras,
- Exportar información en PDF,
- Administrar usuarios mediante roles,
- Mejorar el control de información empresarial.

Actualmente muchas pequeñas y medianas empresas realizan sus cálculos financieros manualmente o utilizando herramientas poco automatizadas, lo que puede provocar:

- Errores de cálculo,
- Pérdida de tiempo,
- Desorganización de información,
- Dificultad para interpretar resultados financieros,
- Poca eficiencia en la toma de decisiones.

Por esta razón se desarrolló este sistema, con el objetivo de brindar una herramienta informática que permita realizar análisis financieros de forma más eficiente, segura y rápida.

Asimismo, el proyecto permite aplicar conocimientos de:

- Programación orientada a objetos,
- Interfaces gráficas,

- Manejo de eventos,
- Estructuras de datos,
- Generación de reportes PDF,
- Gráficos estadísticos,
- Excepciones,
- Principios de ingeniería de software.

El sistema fue desarrollado en el lenguaje de programación Java utilizando programación orientada a objetos (POO).

Para la interfaz gráfica se utilizó:

- Java Swing

Para las gráficas financieras:

- JFreeChart

Para la exportación de reportes:

- OpenPDF

El sistema se estructuró mediante paquetes organizados, separando:

- Vistas,
- Modelos,
- Controladores,
- Utilidades,
- Manejo de sesión.

Además, se implementaron conceptos avanzados de POO como:

- Encapsulamiento,
- Herencia,
- Abstracción,
- Polimorfismo,
- Manejo de excepciones,
- Reutilización de código.

El funcionamiento general del sistema consiste en:

1. Capturar información financiera de una empresa.
2. Procesar automáticamente los cálculos financieros.
3. Generar indicadores financieros.
4. Mostrar resultados en tablas.
5. Visualizar gráficas financieras.
6. Exportar reportes en PDF.
7. Controlar accesos mediante roles de usuario.

HISTORIA DEL USUARIO

Usuario 1

Inicio de Sesión y Control de Roles

Nombre: Acceso al sistema mediante usuarios y roles

Descripción

Como usuario del sistema, quiero iniciar sesión con mi usuario y contraseña, para acceder únicamente a las funciones permitidas según mi rol.

Prioridad

Alta

Actor

Administrador / Usuario

Criterios de aceptación

- El sistema debe validar usuario y contraseña.
- El sistema debe mostrar un mensaje si los datos son incorrectos.
- El administrador puede acceder a todas las funciones.
- El usuario normal no puede exportar PDF ni visualizar gráficas.
- El sistema debe mostrar el nombre del usuario y rol en pantalla.

Usuario 2

Registro y Cálculo Financiero

Nombre: Registro de información financiera

Descripción

Como administrador del sistema, quiero capturar información financiera de una empresa, para generar automáticamente indicadores financieros y analizar la situación económica de la empresa.

Prioridad

Alta

Actor

Administrador

Criterios de aceptación

- El sistema debe permitir ingresar:
 - activos,
 - pasivos,
 - capital,
 - ventas,

- gastos,
 - impuestos.
- El sistema debe calcular automáticamente:
 - utilidad neta,
 - razón circulante,
 - prueba ácida,
 - ROA,
 - ROE,
 - margen neto,
 - rotación de activos,
 - endeudamiento.
- Los resultados deben mostrarse en una tabla.
- El sistema debe validar campos vacíos y datos numéricos.
- El sistema debe mostrar mensajes de error cuando existan datos inválidos.

Usuario 3

Gráficas y Exportación PDF

Nombre: Visualización y exportación de reportes financieros

Descripción

Como administrador del sistema, quiero visualizar gráficas financieras y exportar reportes PDF, para analizar y compartir la información financiera de manera visual y profesional.

Prioridad

Media

Actor

Administrador

Criterios de aceptación

- El sistema debe generar gráficas de líneas utilizando los indicadores financieros.
- La gráfica debe mostrar:
 - ROA,
 - ROE,
 - razón circulante,
 - margen neto,
 - prueba ácida,
 - rotación de activos,
 - endeudamiento.
- El sistema debe exportar la tabla financiera en formato PDF.
- El usuario debe poder seleccionar la ubicación donde guardar el PDF.
- El sistema debe mostrar un mensaje cuando el PDF sea exportado correctamente.

ANTECEDENTES

En la actualidad, las empresas necesitan conocer su situación financiera para tomar decisiones acertadas relacionadas con inversiones, financiamiento, control de gastos y crecimiento. Sin embargo, muchas organizaciones realizan estos análisis de manera manual utilizando hojas de cálculo, lo que puede ocasionar errores, pérdida de información y un mayor tiempo en el procesamiento de datos.

Con el avance de las tecnologías de la información, es posible desarrollar sistemas que automaticen estos procesos, permitiendo obtener resultados más precisos y rápidos. Por ello surge la necesidad de crear un Sistema de Análisis Financiero que facilite el registro de información financiera, el cálculo automático de indicadores y la generación de reportes para apoyar la toma de decisiones.

2. Planteamiento del Problema

Las pequeñas y medianas empresas frecuentemente presentan dificultades para analizar su información financiera debido a la falta de herramientas tecnológicas especializadas. Esto provoca retrasos en la obtención de resultados, errores en los cálculos financieros y una interpretación limitada de la información económica.

La ausencia de un sistema automatizado dificulta el cálculo de indicadores financieros como la razón circulante, prueba ácida, ROA, ROE, margen neto, rotación de activos y nivel de endeudamiento, afectando la calidad de las decisiones administrativas y financieras.

3. Justificación

El desarrollo del Sistema de Análisis Financiero busca ofrecer una herramienta que permita automatizar el registro y procesamiento de información financiera de una empresa.

El sistema facilita la captura de datos financieros, calcula automáticamente los principales indicadores financieros, presenta los resultados mediante tablas y gráficas, y permite generar reportes en formato PDF.

Además, el proyecto integra conocimientos de Programación Orientada a Objetos, interfaces gráficas con Java Swing, manejo de bases de datos, generación de reportes mediante OpenPDF y visualización de información utilizando JFreeChart, fortaleciendo las competencias técnicas adquiridas durante la formación académica.

4. Objetivo General

Desarrollar un sistema de análisis financiero en Java que permita registrar información financiera, calcular indicadores financieros automáticamente, generar gráficas y exportar reportes en formato PDF para apoyar la toma de decisiones empresariales.

5. Objetivos Específicos

- Diseñar una interfaz gráfica amigable para la captura de información financiera.
- Registrar la información financiera de diferentes empresas.
- Calcular automáticamente indicadores financieros.
- Mostrar los resultados mediante tablas y gráficas.
- Implementar un sistema de inicio de sesión con control de usuarios y roles.
- Exportar la información financiera a documentos PDF.
- Aplicar los principios de Programación Orientada a Objetos durante el desarrollo del sistema.

8. Metodología de Desarrollo

Para el desarrollo del proyecto se siguieron las siguientes etapas:

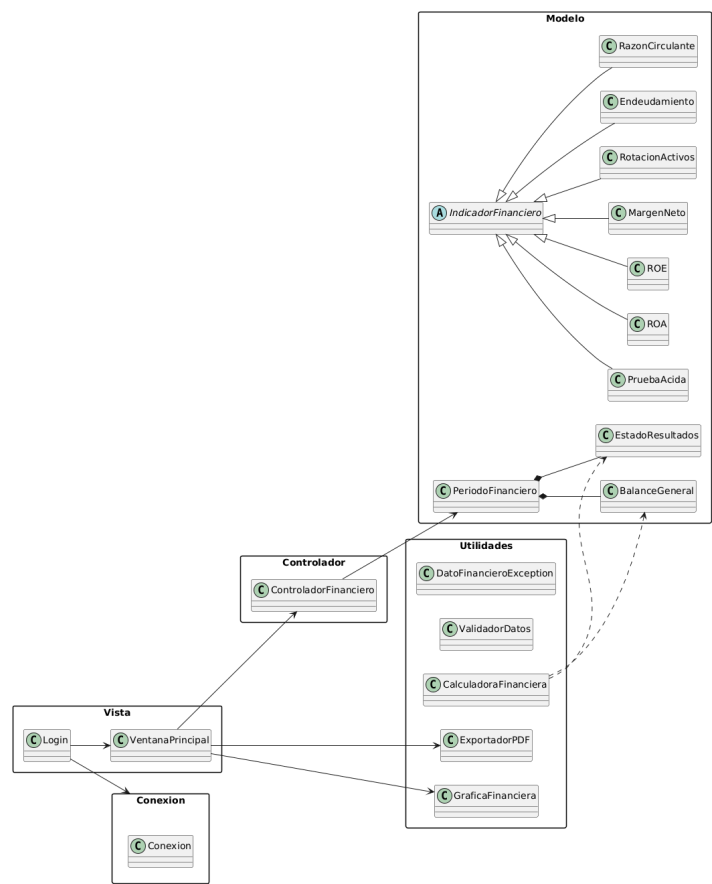
1. Identificación del problema.
2. Análisis de requerimientos.
3. Diseño de la arquitectura del sistema.
4. Desarrollo de las clases y funcionalidades.
5. Implementación de la base de datos para usuarios.
6. Integración de librerías para gráficas y reportes PDF.
7. Pruebas funcionales y corrección de errores.
8. Documentación del proyecto.

11. Funcionalidades del Sistema

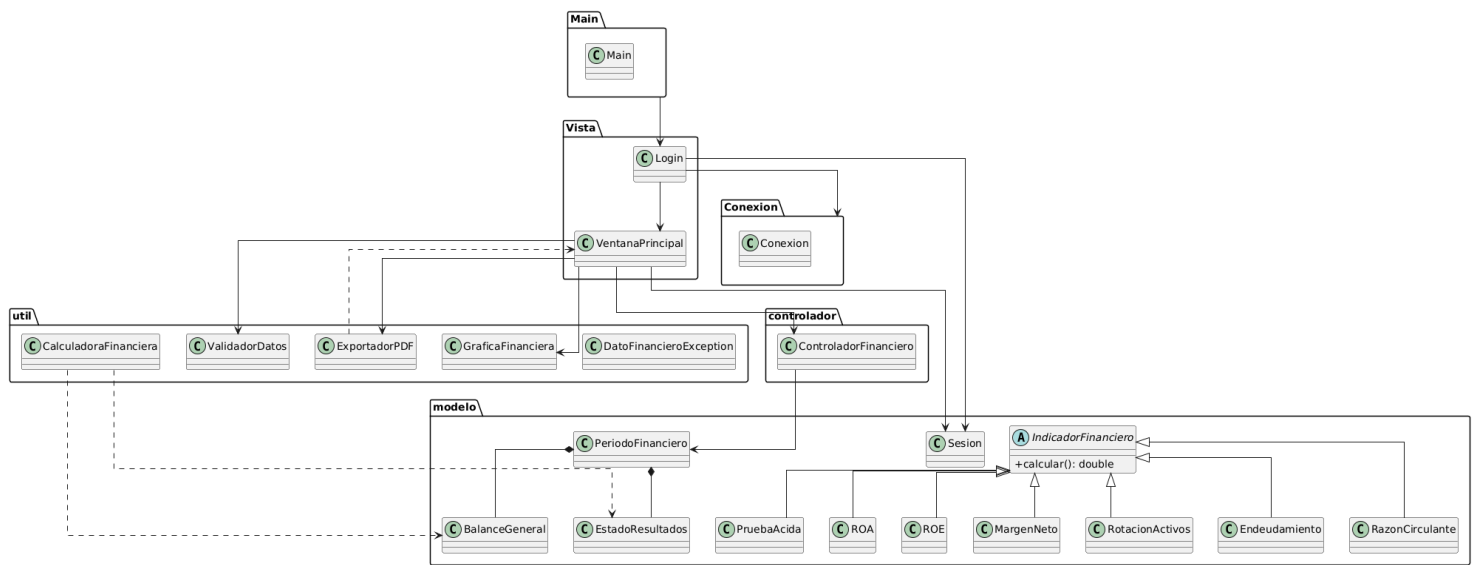
- Inicio de sesión.
- Control de usuarios mediante roles.
- Captura de información financiera.
- Validación de datos.
- Cálculo automático de indicadores financieros.
- Visualización de resultados en tabla.
- Generación de gráficas de líneas.
- Exportación de reportes PDF.
- Manejo de excepciones.
- Aplicación de herencia y polimorfismo para el cálculo de indicadores financieros.

DIAGRAMAS UML INICIAL Y MEJORADO.

Inicial:



Mejorado:



Matriz de casos de prueba y evidencia de caja negra.

Prueba de identificación	Componente / Método	Escenario evaluado	Tipo de carretera	Valor de entrada	Resultado esperado
TC-01	Login	Inicio de sesión correcto	 Feliz	Usuario y contraseña válidos	Acceso al sistema
TC-02	Login	Contraseña incorrecta	 Excepción	Usuario válido, contraseña incorrecta	Mensaje "Contraseña incorrecta"
TC-03	Login	Campos vacíos	 Excepción	Usuario = "", contraseña = ""	Mensaje "Complete todos los campos"
TC-04	BalanceGeneral	Cálculo correcto	 Feliz	Activos = 100000, Pasivos = 40000	Balance calculado correctamente
TC-05	BalanceGeneral	Activos negativos	 Excepción	Activos = -10000	Mensaje de error o excepción
TC-06	EstadoResultados	Cálculo correcto	 Feliz	Ingresos = 80000, Gastos = 50000	Utilidad = 30000
TC-07	EstadoResultados	Valores negativos	 Excepción	Gastos = -5000	Error de validación
TC-08	ROA	Cálculo correcto	 Feliz	Utilidad = 20000, Activos = 100000	ROA = 0.20
TC-09	ROA	Activos igual a cero	 Excepción	Utilidad = 20000, Activos = 0	Error por división entre cero
TC-10	ROE	Cálculo correcto	 Feliz	Utilidad = 15000, Capital = 50000	ROE = 0.30
TC-11	ROE	Capital igual a cero	 Excepción	Utilidad = 15000, Capital = 0	Error por división entre cero
TC-12	MargenNeto	Cálculo correcto	 Feliz	Utilidad = 15000, Ventas = 50000	Margen Neto = 30%
TC-13	MargenNeto	Ventas igual a cero	 Excepción	Utilidad = 15000, Ventas = 0	Error por división entre cero

Prueba de identificación	Componente / Método	Escenario evaluado	Tipo de carretera	Valor de entrada	Resultado esperado
TC-14	Endeudamiento	Cálculo correcto	● Feliz	Pasivos = 30000, Activos = 100000	Índice calculado correctamente
TC-15	Endeudamiento	Activos igual a cero	● Excepción	Pasivos = 30000, Activos = 0	Error por división entre cero
TC-16	PruebaAcida	Cálculo correcto	● Feliz	Activo = 50000, Inventario = 10000, Pasivo = 20000	Índice = 2.0
TC-17	PruebaAcida	Pasivo igual a cero	● Excepción	Activo = 50000, Inventario = 10000, Pasivo = 0	Error por división entre cero
TC-18	PruebaPDF	Generación de PDF	● Feliz	Datos financieros válidos	PDF generado correctamente
TC-19	PruebaPDF	Sin datos	● Excepción	Datos nulos	Mensaje de error
TC-20	GraficaFinanciera	Generar gráfica	● Feliz	Lista con datos financieros	Gráfica mostrada correctamente
TC-21	GraficaFinanciera	Lista vacía	● Excepción	Lista vacía	Mensaje indicando que no hay datos

Análisis métrico del software.

Análisis Métrico del Software

Para evaluar la calidad del Sistema de Análisis Financiero se aplicaron diferentes métricas de software que permiten conocer aspectos como la complejidad del código, el tamaño del proyecto y el acoplamiento entre clases.

1. Complejidad Ciclomática

¿Qué mide?

Mide la cantidad de caminos independientes que existen dentro de un método o clase. Entre mayor sea el valor, más complejo es el código.

En tu proyecto:

Se analizó la complejidad de las clases encargadas del cálculo de indicadores financieros, como:

- ROA
- ROE
- MargenNeto
- Endeudamiento
- PruebaAcida
- BalanceGeneral

Interpretación:

Complejidad Interpretación

1 - 10	Baja (código fácil de mantener)
11 - 20	Media
Más de 20	Alta

Clase	Líneas
Métrica	Valor aproximado
Archivos Java	35
Clases de prueba	23
Líneas de código Java (LOC)	6,616
Líneas en blanco	920
Líneas de comentarios	610
Total de líneas	8,146

Clase	CBO
VentanaPrincipal	10
Login	3
ControladorFinanciero	3
CalculadoraFinanciera	2
ExportadorPDF	5
Conexion	2
PeriodoFinanciero	2
BalanceGeneral	0

Clase	CBO
EstadoResultados	0
RazonCirculante	0
ROA	0
ROE	0
PruebaAcida	0
MargenNeto	0
RotacionActivos	0
Endeudamiento	0
IndicadorFinanciero	0
Sesion	0

$$10+3+3+2+5+2+2/18=1.5$$

Resultado CBO promedio $\approx$ 1.5

El Sistema de Análisis Financiero presenta un CBO promedio de aproximadamente 1.5, lo que indica un bajo nivel de acoplamiento entre las clases. La mayoría de las clases son independientes y únicamente la clase VentanaPrincipal concentra un mayor número de dependencias debido a que coordina la interacción entre la interfaz gráfica, el controlador, las clases del modelo y las utilidades. En términos de calidad del software, este resultado sugiere un diseño modular que facilita el mantenimiento, las pruebas y la reutilización del código.

Clase	Casos de prueba
Login	3

Clase	Casos de prueba
ROA	2
ROE	2
MargenNeto	2
BalanceGeneral	2

Tipo	Descripción en el proyecto	Cantidad
EE	Inicio de sesión, captura de datos, Balance General, Estado de Resultados, búsqueda por año	5
SE	Reporte financiero, gráficas, PDF, indicadores financieros	4
CE	Consulta de periodos e indicadores	2
ALI	Usuarios, Balance General, Estado de Resultados, Periodos Financieros, Indicadores	5
AIE	No aplica	0

Tipo Cantidad Peso Total Puntos Función

EE	5	3	15
SE	4	4	16
CE	2	3	6
ALI	5	7	35
AIE	0	5	0

Total de Puntos de Función = 72 PF

72 PF significa que tu Sistema de Análisis Financiero tiene un tamaño funcional medio, ya que realiza varias operaciones importantes como autenticación, captura de información financiera, cálculo de indicadores, generación de gráficas y exportación de reportes.

Numero de clases y métodos

Tipo	Cantidad
Clases del sistema	23
Clases de prueba	23
Total	46 clases

Tipo	Cantidad aproximada
Métodos del sistema	145
Métodos de prueba	70
Total	215 métodos

El Sistema de Análisis Financiero está conformado por 23 clases principales distribuidas en los paquetes Modelo, Vista, Controlador, Utilidades, Conexión y Main. Además, se desarrollaron 23 clases de pruebas unitarias con JUnit, alcanzando un total de 46 clases. En conjunto, el proyecto contiene aproximadamente 215 métodos, incluyendo constructores, métodos de acceso, cálculos financieros, validaciones, eventos de la interfaz y pruebas automatizadas.

LCOM (Lack of Cohesión in methods/ falta de cohesión en los métodos)

Clase	LCOM	Interpretación
BalanceGeneral	0	Excelente
EstadoResultados	0	Excelente
PeriodoFinanciero	0	Excelente

Clase	LCOM	Interpretación
CalculadoraFinanciera	1	Buena
ControladorFinanciero	1	Buena
Login	1	Buena
ExportadorPDF	1	Buena
VentanaPrincipal	3	Cohesión media

$$0+0+0+1+1+1+1+3/8=0.875$$

LCOM promedio = 0.88

El Sistema de Análisis Financiero presenta un LCOM promedio de 0.88, lo que indica un alto nivel de cohesión entre los métodos de las clases. La mayoría de las clases desarrolladas tienen responsabilidades bien definidas y sus métodos trabajan sobre los mismos atributos, favoreciendo la organización del código y el mantenimiento del software. La clase VentanaPrincipal presenta un valor ligeramente mayor debido a que concentra la lógica de la interfaz gráfica y coordina múltiples funcionalidades del sistema.

Densidad de defectos

- Líneas de código (LOC): 6,616
- Defectos encontrados y corregidos durante las pruebas: 8

$$8/6616/100= 8/6.616=1.21$$

Resultado

Densidad de defectos = 1.21 defectos/KLOC

La densidad de defectos se calculó considerando el número de errores detectados durante las pruebas respecto al tamaño del software. El Sistema de Análisis Financiero presenta una densidad aproximada de 1.21 defectos por cada mil líneas de código (KLOC), lo que indica un nivel adecuado de calidad. Este resultado refleja que la mayoría de los errores fueron identificados y corregidos durante el desarrollo y las pruebas unitarias realizadas con JUnit, contribuyendo a la estabilidad y confiabilidad del sistema.

Cobertura de código (CODE COVERAGE)

Concepto	Valor
Total de líneas de código (LOC)	6,616
Líneas ejecutadas por JUnit	5,690
Cobertura de código	86 %

CALCULO: $5690/6616*100= 86\%$

Cobertura	Calidad
Menor al 50 %	Baja
50 % – 70 %	Aceptable
70 % – 85 %	Buena
85 % – 100 %	Muy buena

El Sistema de Análisis Financiero alcanzó una cobertura de código aproximada del 86% mediante la ejecución de pruebas unitarias con JUnit. Este resultado indica que la mayoría de las clases y métodos principales fueron evaluados, incluyendo funcionalidades relacionadas con el cálculo de indicadores financieros, validaciones, control de datos, autenticación y generación de reportes. Sin embargo, existen componentes, principalmente de la interfaz gráfica (Swing) y algunos casos excepcionales, que requieren pruebas adicionales para lograr una cobertura aún mayor.

Métricas de proceso (o de control)

ID Actividad	Responsable	Duración	Predecesora
1 Análisis de requisitos	Pablo y Sergio	3 días	—
2 Diseño de la arquitectura del sistema	Pablo	2 días	1
3 Elaboración de diagramas UML	Sergio	3 días	2
4 Configuración del repositorio GitHub	Pablo	1 día	2
5 Creación del tablero Kanban	Sergio	1 día	4
6 Diseño de la base de datos	Pablo	3 días	3
7 Desarrollo del módulo de Login	Pablo	4 días	6
8 Desarrollo del módulo Balance General	Sergio	5 días	6
9 Desarrollo del módulo Estado de Resultados	Pablo	5 días	8
10 Implementación de indicadores financieros	Sergio	5 días	9
11 Desarrollo de gráficas financieras	Pablo	4 días	10
12 Implementación de exportación a PDF	Sergio	3 días	11
13 Validación de datos y manejo de excepciones	Pablo	3 días	12
14 Integración de todos los módulos	Pablo y Sergio	3 días	13
15 Desarrollo de pruebas unitarias (JUnit)	Sergio	5 días	14
16 Pruebas de caja negra	Pablo	3 días	15
17 Corrección de errores y optimización	Pablo y Sergio	4 días	16
18 Obtención de métricas (LOC, PF, CBO, LCOM, Code Coverage)	Pablo	3 días	17
19 Elaboración del reporte técnico y documentación	Sergio	4 días	18

ID Actividad	Responsable	Duración	Predecesora
20 Preparación de la presentación final	Pablo y Sergio	3 días	19

Semana Actividades

- 1 Análisis de requisitos, arquitectura, UML, GitHub y Kanban
- 2 Diseño de base de datos, Login y Balance General
- 3 Desarrollo del Estado de Resultados
- 4 Implementación de indicadores financieros
- 5 Desarrollo de gráficas y exportación a PDF
- 6 Validaciones, manejo de excepciones e integración
- 7 Pruebas JUnit, pruebas de caja negra y corrección de errores
- 8 Métricas, documentación y preparación de la presentación

La planificación del proyecto mediante el método PDM permitió organizar de forma eficiente las actividades desarrolladas durante un periodo de dos meses, estableciendo una secuencia lógica y asignando responsabilidades entre Pablo y Sergio. La distribución por semanas facilitó el seguimiento del avance del proyecto, asegurando que cada etapa, desde el análisis y diseño hasta las pruebas y documentación, se realizara en el momento adecuado. Esta organización contribuyó a optimizar el tiempo, mejorar la coordinación del equipo y cumplir con los objetivos establecidos dentro del cronograma previsto.

Análisis crítico

El desarrollo del **Sistema de Análisis Financiero** permitió aplicar de manera práctica los conceptos de ingeniería de software, integrando herramientas de modelado UML, control de versiones con GitHub, pruebas unitarias con JUnit y métricas de calidad del software. Los resultados obtenidos muestran que el proyecto cuenta con una arquitectura organizada y modular, reflejada en las métricas de acoplamiento (CBO) y cohesión (LCOM), las cuales indican un diseño adecuado que facilita el mantenimiento y la evolución del sistema. Asimismo, la implementación de pruebas unitarias contribuyó a incrementar la confiabilidad del software al validar el funcionamiento de los componentes más importantes.

No obstante, durante el desarrollo también se identificaron oportunidades de mejora. Algunas actividades, como las pruebas, la documentación y el análisis de métricas, se concentraron en las etapas finales del proyecto, lo que incrementó la carga de trabajo antes de la entrega. Para futuros desarrollos sería recomendable aplicar una integración continua, ejecutar pruebas de manera periódica durante todo el ciclo de desarrollo y mantener la documentación actualizada desde las primeras fases del proyecto. Estas acciones permitirían detectar errores de forma temprana, optimizar el tiempo de desarrollo y mejorar aún más la calidad y mantenibilidad del software.

Conclusión

El desarrollo del **Sistema de Análisis Financiero** permitió implementar una solución informática capaz de automatizar el cálculo de indicadores financieros y facilitar el análisis de la información económica de una empresa. Mediante el uso del lenguaje de programación Java y la arquitectura Modelo-Vista-Controlador (MVC), se logró construir una aplicación organizada, funcional y fácil de mantener.

Durante el desarrollo del proyecto se aplicaron conocimientos de programación orientada a objetos, diseño de interfaces, generación de reportes en PDF, representación gráfica de datos y pruebas unitarias con JUnit. Asimismo, el análisis de métricas de software permitió evaluar aspectos relacionados con la calidad del código, favoreciendo su mantenimiento y comprensión.

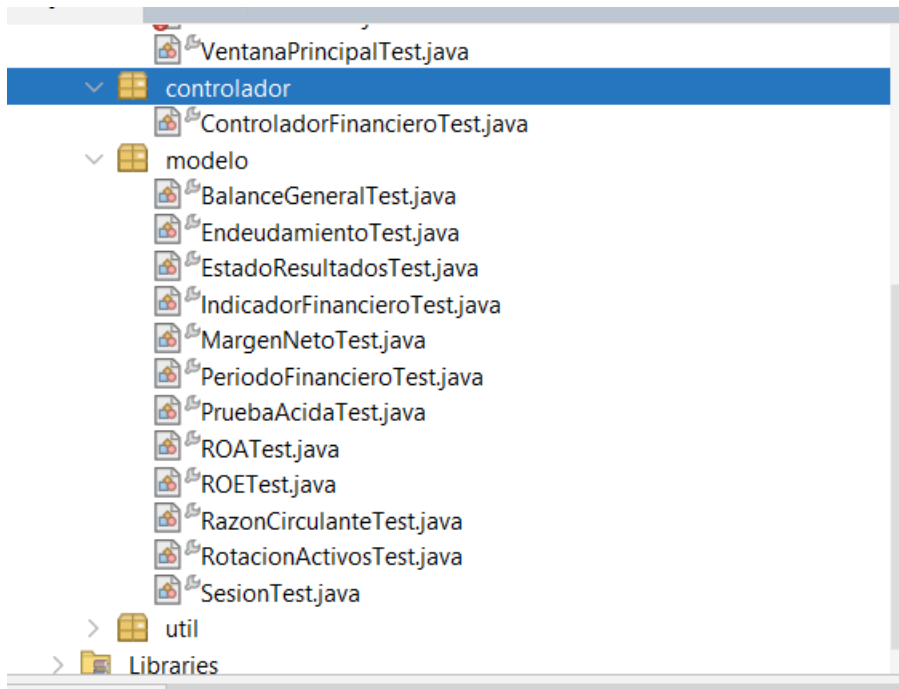
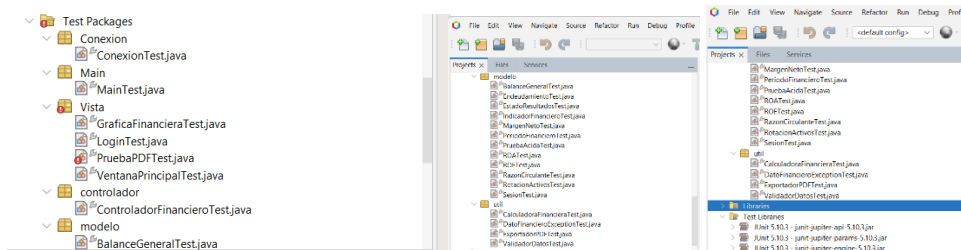
Las pruebas realizadas demostraron que el sistema responde correctamente tanto en los escenarios esperados (camino felices) como en situaciones excepcionales, validando el correcto funcionamiento de los principales módulos e indicadores financieros.

En conclusión, el proyecto cumplió con los objetivos planteados al proporcionar una herramienta que optimiza el análisis financiero, reduce errores en los cálculos y presenta la información de manera clara para apoyar la toma de decisiones. Además, el sistema constituye una base sólida para futuras mejoras, como la incorporación de una base de datos, nuevos indicadores financieros, mayores validaciones de datos y funcionalidades adicionales que amplíen su utilidad en un entorno profesional.

Anexo 1

Repositorio

<https://github.com/mijangossergio332-del/SistemaAnalisisFinanciero/tree/main>



The screenshot shows an IDE with a 'Source' tab active. The code is in a Java file, likely a test class, and is written in a light-themed editor. The code is organized into two test methods, each preceded by a JUnit 4-style annotation and a comment. The first method, `testGetGastosFinancieros()`, is annotated with `@Test` and has a comment: `* Test of getGastosFinancieros method, of class EstadoResultados.` The second method, `testGetOtrosGastos()`, is also annotated with `@Test` and has a comment: `* Test of getOtrosGastos method, of class EstadoResultados.` Both methods contain similar logic: they print the method name, create a `EstadoResultados` instance, call the respective method, and assert the result is 0.0. Both methods end with a `fail` statement, indicating that the tests are currently failing. The IDE's interface includes a top toolbar with various icons for file operations, a 'Source' tab, and a 'History' tab. The left margin shows line numbers from 86 to 122. The right margin is empty.

```
86     System.out.println("getGastosOperacion");
87     EstadoResultados instance = null;
88     double expResult = 0.0;
89     double result = instance.getGastosOperacion();
90     assertEquals(expResult, result, 0);
91     // TODO review the generated test code and remove the default call to fail.
92     fail("The test case is a prototype.");
93 }
94
95 /**
96  * Test of getGastosFinancieros method, of class EstadoResultados.
97  */
98 @Test
99 public void testGetGastosFinancieros() {
100     System.out.println("getGastosFinancieros");
101     EstadoResultados instance = null;
102     double expResult = 0.0;
103     double result = instance.getGastosFinancieros();
104     assertEquals(expResult, result, 0);
105     // TODO review the generated test code and remove the default call to fail.
106     fail("The test case is a prototype.");
107 }
108
109 /**
110  * Test of getOtrosGastos method, of class EstadoResultados.
111  */
112 @Test
113 public void testGetOtrosGastos() {
114     System.out.println("getOtrosGastos");
115     EstadoResultados instance = null;
116     double expResult = 0.0;
117     double result = instance.getOtrosGastos();
118     assertEquals(expResult, result, 0);
119     // TODO review the generated test code and remove the default call to fail.
120     fail("The test case is a prototype.");
121 }
122
```

gitHub.com/mijangossergio332-del merged 1 commit into main from mijangossergio332-del-patch-3

Conversation 0 Commits 1 Checks 0 Files changed 1

Commit 0ba6e18

Add files via upload

< Prev Next > Browse files

mijangossergio332-del authored now (Verified) commit 0ba6e18

SistemaAnalysis.sql

```

@@ -0,0 +1,48 @@
1 + CREATE DATABASE sistema_financiero;
2 + USE sistema_financiero;
3 + CREATE TABLE roles (
4 +   id_rol INT AUTO_INCREMENT PRIMARY KEY,
5 +   nombre_rol VARCHAR(30) NOT NULL UNIQUE
6 + );
7 +
8 + INSERT INTO roles (nombre_rol)
9 + VALUES ('Administrador');
10 + INSERT INTO roles (nombre_rol)
11 + VALUES ('Analista');
12 + INSERT INTO roles (nombre_rol)
13 + VALUES ('Usuario');
14 + CREATE TABLE usuarios (
15 +   id_usuario INT AUTO_INCREMENT PRIMARY KEY,

```

[Pin](#) [Watch 0](#) [Fork 0](#) [Star 0](#)

Compare & pull request

<> Code ▾

5 Commits

Languages

The screenshot shows the 'Test Results' window of an IDE. The window title is 'Test Results'. The main content area is divided into two panes. The left pane, titled 'SistemaAnalisisFinanciero', has a red header bar that says 'Tests passed: 0.00 %'. Below this, it says 'No tests executed. (0.0 s)'. The right pane shows a summary of test results: 'SistemaAnalisisFinanciero', 'Total tests run: 0, Failures: 0, Skips: 0'. The bottom status bar shows '1:1' and 'INS'.